Web System Technologies

IT0049

Module 1

Introduction to MVC

Objectives

- To explore and comprehend one of the fastest tool to develop a web-based system.
- To understand the importance of using a framework.
- To familiarize student on web application framework architecture. concepts, kind, models.
- To understand the flow of MVC.
- To introduce CodeIgniter as an MVC framework.
- To familiarize student on CodeIgniter's features, design and architectural goals.

History

- 1993, the **Common Gateway Interface (CGI)** standard was introduced.

- 1995, fully integrated server/language development environments first emerged and new web-specific languages were introduced.

- In the late 1990s, mature, "full stack" frameworks began to appear.

Web Application Framework

A **web application framework (WAF)** is a software framework that is designed to support the development of web applications including web services, web resources, and web APIs.

Web Application Framework

Web frameworks provide a standard way to build and deploy web applications on the World Wide Web.

Web Application Framework

The framework aims to alleviate the overhead associated with common activities performed in web development.

For example, many frameworks provide libraries for database access, templating frameworks and session management, and they often promote code reuse.

Types of framework architectures

- Model–view–controller (MVC)
- Push-based vs. pull-based
- Three-tier organization

Most web application frameworks are based on the model–view–controller (MVC) pattern.

Framework applications

Frameworks are built to support the construction of internet applications based on a single programming language, ranging in focus from general purpose tools to native-language programmable packages built around a specific user application.

General-purpose website frameworks

Web frameworks must function according to the architectural rules of browsers and web protocols such as HTTP, which is a stateless protocol.

- Server-side scripting
- Client-side scripting

Server-side

Server-side refers to operations that are performed by the server in a client–server relationship in a computer network.

Server-side scripting is a technique used in web development which involves employing scripts on a web server which produce a response customized for each user's (client's) request to the website. The alternative is for the web server itself to deliver a static web page.

Server-side

Typically, a server is a computer application, such as a web server, that runs on a remote server, reachable from a user's local computer, smartphone, or other device.

Server-side

In the context of the World Wide Web, commonly encountered server-side computer languages include

- C# or Visual Basic in ASP.NET environments
- Java
- Perl
- PHP
- Python
- Ruby
- Node.js
- Swift

Server-side

However, web applications and services can be implemented in almost any language, as long as they can return data to standards-based web browsers (possibly via intermediary programs) in formats which they can use.

Client-side

Client-side refers to operations that are performed by the client in a client–server relationship in a computer network

Client-side scripting is the changing of interface behaviors within a specific web page in response to mouse or keyboard actions, or at specified timing events.

Client-side

Typically, a client is a computer application, such as a web browser, that runs on a user's local computer, smartphone, and connects to a server as necessary.

Client-side scripting

In the context of the World Wide Web, commonly encountered computer languages which are evaluated or run on the client side include:

- Cascading Style Sheets (CSS)
- HTML
- JavaScript

Features of Web Application Framework

Frameworks typically set the control flow of a program. The common features of WAF are the following :

- Web Template System
- Caching
- Security
- Database access, mapping and configuration
- URL Mapping
- Web Services
- Web Resources

Introduction to MVC

MVC

MODEL VIEW CONTROLLER (MVC) is a software design pattern commonly used for developing user interfaces that divides the related program logic into three interconnected elements.

Traditionally used for desktop graphical user interfaces (GUIs), this pattern has become popular for designing web applications.

HISTORY

- 1979 - Trygve Reenskaug introduced MVC into Smalltalk-79.
- 1980s - Jim Althoff and others implemented a version of MVC for the Smalltalk-80 class library. Evolved into other variants such as hierarchical model–view–controller (HMVC), model–view–adapter (MVA), model–view–presenter (MVP), model–view–viewmodel (MVVM).
- 1996 - The use of the MVC pattern in web applications exploded in popularity after the introduction of NeXT's WebObjects.

PRINCIPLES OF MVC

Web applications that uses the WebForm approach commingle :

- Database Code
- Page Design Code
- Control Flow Code

Unless these elements are separated, larger applications become difficult to maintain.

Components of MVC

- Model – Data Management
- View – User Interface
- Controller – Orchestrates the operation

Model

- Is a part of the application that is responsible for retrieving data from the database.

- Converting it into objects

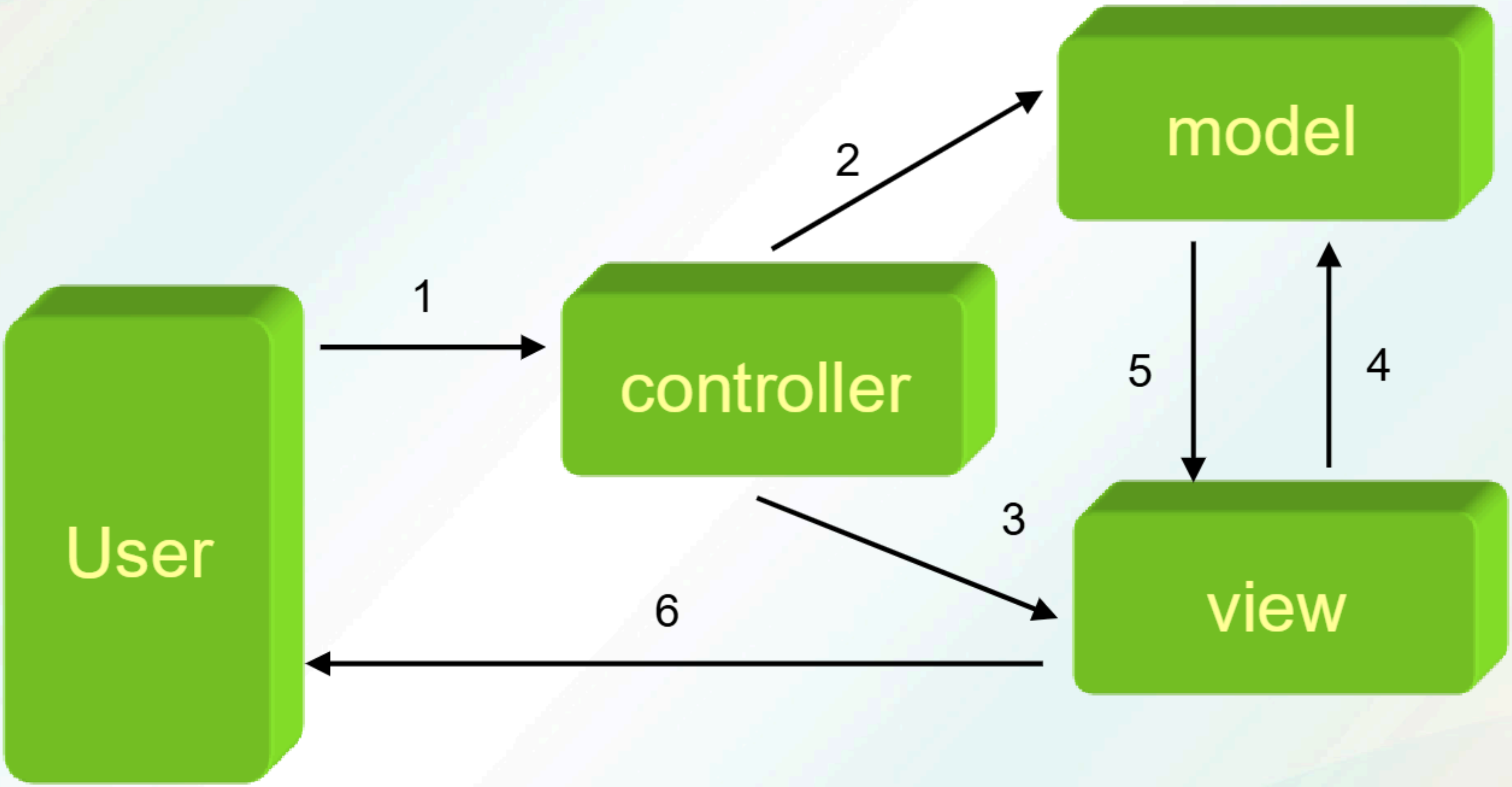
View

- User interface
- Chart, diagram or table
- Presentation of the model
- Data retrieved

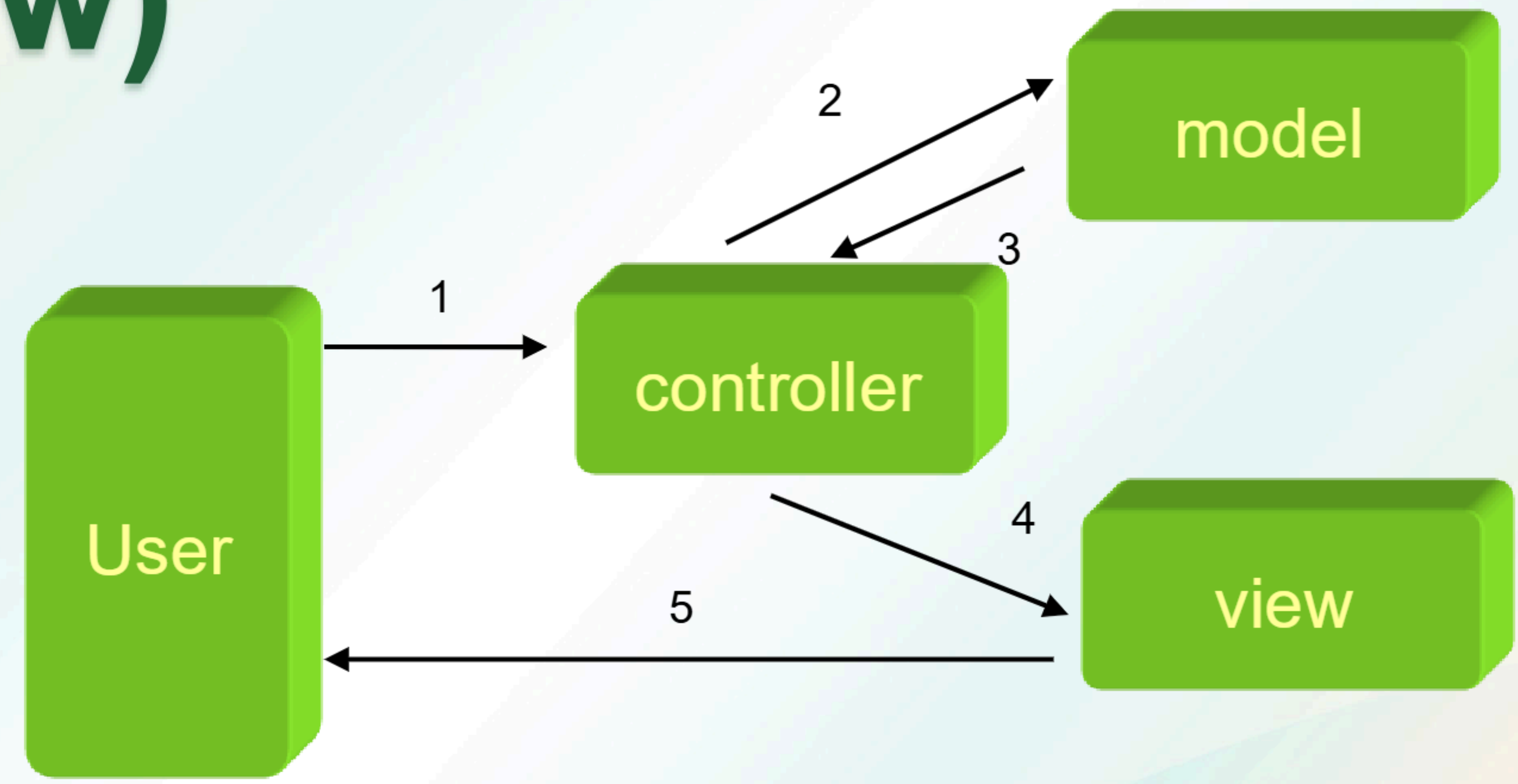
Controller

- Orchestrates the operation
- Validates user input
- Calling the model
- Choosing which view to render
- Handles data to be displayed

MVC Pattern (Model 2)



MVC Pattern (Passive View)



MVC AS GUI FRAMEWORKS

- XPages
- Cocoa framework
- GNUstep
- GTK+
- JFace.
- Microsoft ASP.NET
- Microsoft Composite UI Application Block

MVC AS GUI FRAMEWORKS

- Java Swing.
- Apache Pivot.
- Adobe Flex
- Visual FoxExpress
- Crank Storyboard Suite
- Eiffel

MVC AS WEB FRAMEWORKS

- ABAP Objects
- ActionScript
- C++
- CMFL
- Flex
- Groovy
- JavaScript
- .NET
- PERL
- PHP
- Phyton
- Ruby
- Smalltalk
- XML

Goals of MVC

- Simultaneous development
- Code reuse

Pros

- Simultaneous development
- High cohesion
- Loose coupling
- Ease of modification
- Multiple views for a model
- Testability

Cons

- Code navigability
- Multi-artifact consistency
- Undermined by inevitable clustering
- Excessive boilerplate
- Pronounced learning curve
- Lack of incremental benefit

Introduction to CodeIgniter

CodeIgniter

CodeIgniter is an open-source web framework, for creating web sites with PHP and is loosely based on model–view–controller (MVC) development pattern.

CodeIgniter can be also modified to use Hierarchical Model View Controller (HMVC).

CodeIgniter at a Glance

- Is an Application Framework
- Free
- Light weight
- Fast
- Uses M-V-C
- Generates clean URLs
- Extensible
- Does Not Require a Template Engine
- Thoroughly Documented

CodeIgniter MVC

MVC is a software approach that separates application logic from presentation. In practice, it permits your web pages to contain minimal scripting since the presentation is separate from the PHP scripting.

- The **Model** represents your data structures.
- The **View** is the information that is being presented to a user.
- The **Controller** is the *intermediary* between the Model, the View, and any other resources.

CodeIgniter Features

- Model-View-Controller Based System
- Extremely Light Weight
- Full Featured database classes with support for several platforms.
- Query Builder Database Support
- Form and Data Validation
- Security and XSS Filtering
- Session Management

CodeIgniter Features

- Email Sending Class. Supports Attachments, HTML/Text email, multiple protocols (sendmail, SMTP, and Mail) and more.
- Image Manipulation Library (cropping, resizing, rotating, etc.). Supports GD, ImageMagick, and NetPBM
- File Uploading Class
- FTP Class
- Localization
- Pagination

CodeIgniter Features

- Data Encryption
- Benchmarking
- Full Page Caching
- Error Logging
- Application Profiling
- Calendaring Class
- User Agent Class
- Zip Encoding Class

CodeIgniter Features

- Template Engine Class
- Trackback Class
- XML-RPC Library
- Unit Testing Class
- Search-engine Friendly URLs
- Flexible URI Routing
- Support for Hooks and Class Extensions
- Large library of “helper” functions

Design and Architectural Goals

CodeIgniter's goal is to obtain maximum performance, capability, and flexibility.

CodeIgniter was created with the following objectives:

- Dynamic Instantiation
- Loose Coupling
- Component Singularity